



本小节内容

C 文件概述

文件指针介绍

C 文件概述

程序执行时就称为进程，进程运行过程中的数据均在内存中。需要存储运算后的数据时，就需要使用文件。这样程序下次启动后，就可以直接从文件中读取数据。（不像我们之前的程序，每次运行都需要手动输入数据）。

文件是指存储在外部介质（如磁盘、磁带）上的数据集合。操作系统（Windows、Linux、Mac 等）是以文件为单位对数据进行管理的，如下图所示。

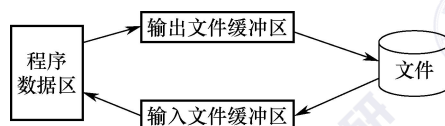


图 操作系统中的文件

C 语言对文件的处理方法如下。

缓冲文件系统：系统自动地在内存区为每个正在使用的文件开辟一个缓冲区。用缓冲文件系统进行的输入/输出称为高级磁盘输入/输出。

非缓冲文件系统：系统不自动开辟确定大小的缓冲区，而由程序为每个文件设定缓冲区。用非缓冲文件系统进行的输入/输出称为低级输入/输出。

下面介绍缓冲区原理。

缓冲区其实**就是一段内存空间**，分为读缓冲、写缓冲。C 语言缓冲的三种特性如下。

(1) **全缓冲：**在这种情况下，当填满标准 I/O 缓冲后才进行实际 I/O 操作。全缓冲的典型代表是对磁盘文件的读写操作。

(2) **行缓冲：**在这种情况下，当在输入和输出中遇到换行符时，将执行真正的 I/O 操作。这时，我们输入的字符先存放到缓冲区中，等按下回车键换行时才进行实际的 I/O 操作。典型代表是标准输入缓冲区（stdin）和标准输出缓冲区（stdout）。

(3) **不带缓冲：**也就是不进行缓冲，标准出错情况（stderr）是典型代表，这使得出错信息可以直接尽快地显示出来（无需掌握，考研不考）。

文件指针介绍

打开一个文件后，我们会得到一个 FILE*类型的文件指针 fp，然后通过该文件指针对文件进行操作。FILE 是一个结构体类型，其具体内容如下所示：

```
struct _iobuf {
```



```
char *_ptr; //下一个要被读取的字符地址
int  _cnt; //剩余的字符,若是输入缓冲区,则表示缓冲区中还有多少个字符未被读取
char *_base; //缓冲区基地址
int  _flag; //读写状态标志位
int  _file; //文件描述符
int  _charbuf;
int  _bufsiz; //缓冲区大小
char *_tmpfname;
};

typedef struct _iobuf FILE;
```

FILE *fp; //在 main.c 写了 FILE *fp; 然后 ctrl+左键就可以跳转到上面的结构体类型定义位置

fp 是一个指向 FILE 类型结构体的指针变量。可以使 **fp** 指向某个文件的结构体变量,从而通过该结构体变量中的文件信息来访问该文件。

Windows 操作系统下的 FILE 结构体与 Linux 操作系统, Mac 操作系统下的 FILE 结构体中的成员变量名是不一致的,但是其原理可以互相参考。